3D 沉浸式年会抽奖系统 —— 基于多模型协同的 AI 编程实践报告

导言：作为一名在校的大四学生，在此次项目中，我没有选择传统的“手敲每一行代码”模式，而是将自己定位为“产品经理 + 资源整合者”。我通过构建 AI 工具矩阵，将核心精力集中在**需求拆解、技术选型把控以及真实业务场景（抽奖组织者）的 UX 优化**上。

(注：我的 GitHub 仓库中特意保留了 index-v2 到 index-v6 的历史版本文件，以真实还原我与 AI 的渐进式协作过程。)

一、 AI 工具矩阵与算力调度 (Tools & Orchestration)

在本项目中，我建立了一个多模型协同的开发网络，以发挥不同 AI 的特长：

- **前期战略大脑**：**Gemini / 豆包** (用于横向对比试题选项，梳理产品核心痛点与商业价值)。
- **核心工程架构**：**Cursor** (依托其 @Codebase 上下文能力进行工程环境搭建与局部代码精修)。
- **代码生成基座模型**：
  - **Claude 3.5 Sonnet**：主导前中期的 3D 视觉动效生成与 HTML/CSS 架构（其对前端视觉的理解力最佳）。
  - **Gemini**：作为 Cursor 算力降级或上下文混乱时的跨端备用方案，主导后期复杂业务逻辑（状态管理、排错修复）的闭环重构。

二、 工作流复盘：从“需求破局”到“渐进式重构” (核心工作流)

我的工作流并非让 AI“一键生成”，而是经历了**“MVP 验证 -> 视觉升维 -> 业务闭环”**的严密协作过程（详见仓库文件演进）：

1. 核心骨架搭建与基础动效 (对应 v1-v2 阶段)

我刻意压制了 AI 试图一次性写完复杂逻辑的倾向。初期仅要求 AI 使用 Vanilla JS 搭建最原始的“数组随机抽取”引擎，随后通过调整 Prompt，将“不够震撼”等非技术语言翻译为具体的 CSS 术语，落实基础的弹窗与背景。

2. 视觉张力突破：3D 球体与爆炸特效 (对应 v3-v4 阶段)
确认底层引擎无误后，我开始指挥 AI 进行视觉升维。放弃传统的 2D 列表，引入了基于原生 CSS3 硬件加速的 3D 标签云（Tag Cloud）自转效果，并结合 cubic-bezier 设计了极具张力的“开奖回弹+彩带喷射”特效。

3. 立足组织者视角的业务重构 (对应 v5-v6 阶段)
纯视觉的抽奖在真实场景中是脆弱的。最后阶段，我跳出代码层面，扮演“产品经理”，要求 AI 在不破坏 3D 场景的前提下，引入前端状态管理，构建了左右两侧的数据管控面板，完成最终的商业级交付。

三、 难点分析与 AI 纠偏记录 (Prompt Engineering)

在与 AI 协作中，最大的挑战是**“遏制 AI 的幻觉与过度工程”**。以下是我解决的两大核心难题：

难点 1：如何落实“非技术向的视觉需求”？并防止 AI“过度工程”？

- **需求挑战**：为了提升交互体验，我希望实现“3D 名字悬浮球体”与“震撼的开奖弹窗”。但题目中给出的反馈（如“弹窗不够震撼”、“高光再红一点”）是非常主观的非技术描述，AI 很难直接理解其背后的视觉构成。
- **AI 的翻车点 (初次尝试)**：
    1. **技术选型失控**：对于“3D 球体”，AI 默认给出了引入 Three.js 或 WebGL 库的方案。我深知对于一个轻量级的 H5 活动页来说，引入重型 3D 库是严重的“过度工程”，会极大增加项目臃肿度与后续维护成本。
    2. **视觉表达廉价**：对于“震撼感”，AI 仅给出了生硬的纯红背景加上最基础的 fade-in 渐显动画，视觉效果极其廉价，毫无张力。
- **我的纠正过程 (Prompt 调优)**：
  我果断叫停了 AI，并将模糊的“主观感觉”降维翻译为**精准的技术参数约束**，下达了重构指令：
  > “【视觉重构严格要求】：
  > 1. 绝对禁止使用 Three.js 等任何 3D 渲染库！必须使用原生 CSS3 的 preserve-3d 与 JS 原生三角函数（黄金比例螺旋算法）计算球面坐标。

> 2. 废弃简单的 fade-in 动画。为了实现'震撼'的开奖效果，请增加强烈的物理弹簧动效（transform: scale(0.1) 到 1.2 再回弹的 cubic-bezier）；背景加入深色半透明遮罩（backdrop-filter: blur(10px)）以聚焦视线；并引入轻量级库 canvas-confetti 实现开奖瞬间的满屏彩带喷射。"

- **最终结果**：在严格的边界约束下，AI 仅用不到 60 行原生 JS 结合硬件加速，就完美模拟了 3D 视差，兼顾了极高的视觉冲击力与工程极简性；同时，通过将主观感觉精准转化为 cubic-bezier、backdrop-filter 以及粒子特效等具体代码，AI 成功输出了极具张力和高级感的开奖动效。

难点 2：AI 布局重构导致的 UI 文字重叠 BUG (跨模型纠偏方案)

- **遇到难题**：在项目推进至 v5 阶段，当我要求 AI 将抽奖界面"全屏化"并引入左右两侧的控制面板时，Cursor 生成的代码破坏了原有的 3D 坐标参考系。导致页面刷新后，所有的"名字标签（UI 文字）"全部重叠、堆砌在屏幕的左上角或正中央，原本优雅的 3D 悬浮球体瞬间坍塌成了一坨乱码。
- **我的纠正过程 (多模型上下文迁移)**：
  我没有盲目依赖 Cursor 在原文件上反复点击"Fix Bug"（这往往会导致 AI 产生更严重的逻辑幻觉）。我迅速提炼了问题的技术本质——**这是 CSS 绝对定位与 JS 动态计算视口中心（Viewport Center）发生冲突导致的。**随后，我将出 BUG 的核心代码提取，跨平台切换至 **Gemini 网页端**，并下达了精准的技术诊断与局部重构指令：
  > "当前在执行全屏化布局后，名字节点出现了严重的 UI 重叠。请重点排查 JS 中计算球心坐标的逻辑，确保其基于动态的视口尺寸；同时检查 CSS 中 .tag 元素的 position: absolute; top: 50%; left: 50%; 以及 translate3d 的映射是否正确。修复此重叠问题后，请顺便在 JS 中新增 winnersList 状态数组，实现完整的库存扣减与右侧抽屉渲染。"
- **最终结果**：Gemini 精准指出了旧代码中由于缺少 translate(-50%, -50%) 居中对齐以及视口动态响应导致的坐标塌陷问题，并给出了修复后的业务闭环代码。我经过 Code Review 后采用"人工接管"全选覆盖至本地项目。这不仅秒解了恶性的 UI 重叠 BUG，还顺利完成了复杂状态管理的迭代，成功化解了单一工具上下文混乱的危机。

四、 加分项回应：立足“抽奖组织者”视角的 UX 体验优化

如果站在**真实的年会主办方（HR/主持人）**的角度，系统若仅有 3D 动画，会面临“奖项超发”、“忘记谁中了什么奖”的灾难性业务痛点。

为此，我利用 AI 辅助构建了完整的**商业级数据闭环**，极大提升了目标用户的体验：

1. **防止超卖的库存管控 (防呆设计)**：界面左侧悬浮毛玻璃奖项面板。实现了“切换奖项 -> 开始抽取 -> 对应奖项库存自动减 1”的闭环。若库存归零，系统会自动拦截抽取请求并置灰卡片，防止奖品超发。
2. **数据沉淀与追溯看板**：右上角增加“ 获奖名单”入口。开奖数据不仅一闪而过，还会沉淀入底层的 winnersList 数组，并在右侧的侧滑抽屉中按奖项分组动态展示。这彻底解放了主持人的记忆负担，方便后续复盘与核销发奖。

五、 总结与展望

通过本次完整的项目交付，我深刻体会到：**在 AI 时代，代码编写本身正在被加速商品化。开发者真正的护城河，在于“对业务真实痛点的洞察力”、“技术选型时的审美与克制”，以及“调度多端 AI 算力解决实际问题的整合能力”。**